

Chat-Zusammenfassung: Programmcode auf Webseiten einbetten

Syntaxhighlighting, Zeilennummern, Scrollbereiche, UTF-8 und Modernisierung einer ESP32-Unterrichtsseite

Erstellt für Andreas - Thema: Code-Darstellung für Unterrichtsseiten, insbesondere ESP32/MicroPython-Seiten. Stand: 25.06.2026.

1. Ziel des Chats

Ausgangspunkt war die Frage, wie Programmcode auf einer Webseite eingebettet und mit Syntaxhighlighting dargestellt werden kann. Daraus entstand ein kleines, wiederverwendbares System für Unterrichtsseiten: externe Code-Dateien werden geladen, farbig hervorgehoben, mit Zeilennummern versehen und in einem begrenzten Scrollbereich angezeigt.

Baustein	Entscheidung im Chat
Syntaxhighlighting	Prism.js
Code aus Datei laden	fetch("main.py") bzw. data-src="blink.py.txt"
Zeilennummern	Prism-Plugin line-numbers
Lange Dateien	max-height + overflow:auto
Lokales Testen	python -m http.server 8000
Provider-Problem	.py-Dateien als .py.txt ausliefern

2. Grundsystem mit Prism.js

Für längere Quelltexte wurde empfohlen, den Code nicht direkt in HTML zu kopieren, sondern als externe Datei zu laden. Dadurch bleibt die Python-Datei eine echte Arbeitsdatei und die Webseite zeigt nur eine Darstellung davon.

HTML-Grundstruktur

```
<pre class="line-numbers codebereich">  
  <code id="codefenster" class="language-python"></code>  
</pre>
```

JavaScript zum Nachladen einer Datei

```
fetch("main.py")  
  .then(response => {  
    if (!response.ok) {  
      throw new Error("HTTP-Fehler " + response.status +  
        " beim Laden von " + response.url);  
    }  
    return response.text();  
  })  
  .then(code => {  
    const codefenster = document.getElementById("codefenster");  
    codefenster.textContent = code;  
    Prism.highlightElement(codefenster);  
  })  
  .catch(error => {  
    document.getElementById("codefenster").textContent =  
      "Fehler beim Laden der Datei: " + error;
```

```
});
```

3. Lokales Testen: kein Doppelklick auf index.html

Ein wichtiger Fehler war: Beim lokalen Öffnen per file:// blockiert der Browser häufig fetch(). Die Lösung ist ein kleiner lokaler Webserver.

```
cd /d D:\Pfad\zum\Projektordner
"D:\Program Files\Python314\python.exe" -m http.server 8000
```

Danach wird die Seite im Browser über `http://localhost:8000/` geöffnet. Der Modulname ist wichtig: Er heißt `http.server` mit Punkt, nicht `http_server`.

Beendet wird der laufende Server im Terminalfenster mit Strg + C. Falls der alte Server nicht gefunden wird oder Port 8000 belegt ist, kann z. B. Port 8080 genutzt werden.

```
"D:\Program Files\Python314\python.exe" -m http.server 8080
```

4. Zeilennummern

Für Verweise im Unterricht sind Zeilennummern sehr nützlich. Prism.js bietet dafür das Plugin `line-numbers`. Die Klasse `line-numbers` gehört an das `pre`-Element, nicht an `code`.

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/prismjs/plugins/line-numbers/prism-line-numbers.css">

<pre class="line-numbers"><code class="language-python"></code></pre>

<script src="https://cdn.jsdelivr.net/npm/prismjs/plugins/line-numbers/prism-line-numbers.min.js"></script>
```

5. Lange Dateien in einem begrenzten Bereich anzeigen

Für lange Programme wurde ein Scrollbereich gewünscht: etwa 10 sichtbare Zeilen, der Rest scrollbar. Das wird allein mit CSS gelöst.

```
pre.codebereich {
  line-height: 1.5;
  max-height: 15em;      /* ca. 10 sichtbare Zeilen */
  overflow: auto;

  padding: 16px 16px 16px 56px;
  border-radius: 8px;
  border: 1px solid #d8dee9;
}

code {
  font-family: Consolas, "Courier New", monospace;
  font-size: 15px;
}
```

Faustregel: `max-height` = gewünschte Zeilenzahl mal `line-height`. Bei `line-height: 1.5` entspricht `max-height: 15em` ungefähr 10 Zeilen.

6. Inline-Code im Fließtext

Kurze Codebestandteile im Erklärungstext werden nicht als großer Block dargestellt, sondern direkt mit `code` im Absatz.

```
<p>
  Die LED wird mit <code class="language-python">led.value(1)</code> eingeschaltet.
</p>
```

Für HTML-Beispiele müssen die spitzen Klammern maskiert werden, sonst interpretiert der Browser den Code als echtes HTML.

```
<code class="language-html">&lt;h1&gt;Hallo&lt;/h1&gt;</code>
```

7. UTF-8-Problem der alten Seite

Die alte Seite enthielt ursprünglich zwei widersprüchliche Zeichensatzangaben: einmal UTF-8 und später Windows-1252. Dadurch konnten Werkzeuge die Seite nicht zuverlässig lesen.

```
<meta charset="UTF-8">
...
<meta http-equiv="Content-Type" content="text/html; charset=Windows-1252">
```

Die Lösung war: die alte Windows-1252-Zeile entfernen und die Datei wirklich als UTF-8 speichern. Danach war die Seite lesbar. Empfohlener Kopf:

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>MicroPython ESP32 D1 R32/mini</title>
  <link rel="stylesheet" type="text/css" href="../../mystyle.css">
  <link rel="stylesheet" type="text/css" href="../../tools/prism/my_prism.css">
  <script type="text/javascript" src="../../tools/prism/prismload.js" id="prismload.js"></script>
</head>
```

8. Modernisierung der ESP32-Unterrichtsseite

Aus den hochgeladenen Dateien micropython.html, mystyle.css, my_prism.css und prismload.js wurde eine modernisierte ZIP erstellt. Ziel war nicht, die alte Seite inhaltlich zu zerstören, sondern die gewachsene Werkstatt aufzuräumen.

- alte oder kaputte Anführungszeichen in Attributen bereinigt
- lang="de" und viewport ergänzt
- viele Inline-Styles durch CSS-Klassen ersetzt
- Codeblöcke mit data-src eingeführt
- Codebereiche auf ca. 10 sichtbare Zeilen begrenzt
- fehlende alt-Texte bei Bildern ergänzt
- Originaldateien im Ordner original/ abgelegt

Neues Schema für Codeblöcke

```
<pre class="line-numbers codebereich">
  <code class="language-python" data-src="blink.py.txt"></code>
</pre>
```

9. Warum .py.txt sinnvoll ist

Der Provider lädt .py-Dateien nicht mehr nach. Deshalb wurden die Python-Dateien mit der Endung .txt versehen. Das ist für Unterrichtsseiten eine saubere Lösung: Der Server behandelt sie als harmlose Textdateien, die Seite stellt sie trotzdem als Python-Code dar.

```
blink.py.txt
pwm.py.txt
adc.py.txt
i2c-scan.py.txt
```

Auf der Webseite kann weiterhin der didaktisch richtige Name angezeigt werden, also z. B. blink.py, während intern blink.py.txt geladen wird.

10. Empfohlene Projektstruktur

Für zukünftige Seiten ist eine klare Struktur sinnvoll. Dadurch bleiben HTML, CSS, JavaScript und Quelltexte sauber getrennt.

```
projektordner/  
├── index.html  
├── style.css  
├── script.js  
├── README.md  
└── programme/  
    ├── blink.py.txt  
    ├── pwm.py.txt  
    ├── adc.py.txt  
    └── i2c-scan.py.txt
```

11. Didaktische Bewertung

Das neue System eignet sich gut für Unterrichtsseiten, weil es drei Dinge zusammenbringt: kurze Erklärungstexte, echte Quelltextdateien und klare Verweise über Zeilennummern. Aus "schaut mal irgendwo im Programm" wird "schaut in Zeile 17". Das ist deutlich präziser.

Die alten Seiten hatten vor allem die Funktion eines schnellen persönlichen Einstiegs. Das war damals völlig sinnvoll. Heute kann daraus schrittweise eine didaktisch saubere Projektseite werden: sichtbare Linktexte, alt-Texte, externe Code-Dateien, Scrollbereiche und einheitliche Karten.

12. Merkliste

- HTML-Dateien eindeutig als UTF-8 speichern und nur eine Charset-Angabe verwenden.
- Lokales Testen immer über `python -m http.server`, nicht per Doppelklick.
- Prism.js: Sprache über `language-python` angeben.
- Zeilennummern: `class="line-numbers"` an `pre` setzen.
- Lange Programme: `max-height` und `overflow:auto` verwenden.
- Provider blockiert `.py`? Dann `.py.txt` verwenden und trotzdem als Python highlighten.
- Bilder mit alt-Text versehen; Links sollten sichtbaren Text haben.

Ende des Dokuments.